LECTURE 6

Neural Networks: Hidden Layers & Backpropagation

Instructor Manual

Part 1: why hidden layers — one neuron's straight line, the XOR wall, non-linear activations and learned feature maps
Part 2: backpropagation by hand — one forward and one backward pass through a 2–2–1 sigmoid network

AK DHAMIJA, Researcher & Instructor, AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

Unit 4: Simulation Strategies in Warfare — AI & Machine Learning in Warfare

This manual contains: the 45-minute session plan with timings; a slide-by-slide teaching guide for all 40 slides; key messages, facilitation tips, anticipated questions with model answers, misconceptions and in-class activities; live demonstrations with the interactive studio; consistency notes on the source material; the complete quiz answer key with rationale and scoring rules; and the assignment with marking scheme and full model solutions.

Contents

1 Lecture at a Glance **2**

2 Session Plan (45 Minutes) **3**

3 Slide-by-Slide Teaching Guide **4**

3.1 1. Overview (slides 2–3, about 1.5 min) . . . 4
3.2 2. One Neuron (slides 4–7, about 3 min) . . . 4
3.3 3. Wide Is Not Enough (slides 8–9, about 1.5 min) . . . 4
3.4 4. Non-Linearity (slide 10, about 1 min) . . . 4
3.5 5. XOR Solved (slides 11–12, about 1.5 min) . . . 4
3.6 6. Feature Maps (slides 13–20, about 6 min) . . . 5
3.7 7. Backprop: Set-Up (slides 21–25, about 4 min) . . . 5
3.8 8. Forward Pass (slides 26–29, about 3 min) . . . 5
3.9 9. Backward Pass (slides 30–35, about 4.5 min) . . . 5
3.10 10. Update & Check (slides 36–38, about 2.5 min) . . . 6
3.11 11. Summary (slides 39–40, about 1.5 min) . . . 6

4 Key Messages **6**

5 Facilitation Tips **6**

6 Anticipated Questions and Model Answers **7**

7 Common Misconceptions **7**

8 In-Class Activities and Discussion Prompts **7**

9 Run It Live: Studio Demonstrations **8**

10 Quiz Administration and Answer Key **9**

11 Assignment: Marking Scheme and Model Solutions **12**

11.1 Problem 1: Single-neuron boundaries and the XOR wall (20 marks) . . . 12
11.2 Problem 2: A ReLU hidden layer: forward pass and collapse (20 marks) . . . 13
11.3 Problem 3: A full sigmoid forward pass (20 marks) . . . 13
11.4 Problem 4: The backward pass (20 marks) . . . 14
11.5 Problem 5: One update and the vanishing gradient (20 marks) . . . 14
11.6 Problem 6 (bonus): Parameter counting and depth (20 marks) . . . 15

1 Lecture at a Glance

Lecture	6 — Neural Networks: Hidden Layers & Backpropagation
Duration	45 minutes: 5 min opening Q&A, 30 min teaching, 10 min auto-quiz
Slides	40 (Beamer PDF, 16:9), 11 sections
Prerequisite	Lecture 5; students should have attempted its quiz and assignment.
Student materials	`Lecture6_Slides_Beamer.pdf`; `Lecture6_Reference_Text.pdf`; `Lecture6_Quiz.html`; `Lecture6_Answers_Assignment.html` (release after the quiz)
Interactive studio	`Lecture6_Neural_Network_Lab.html` (live demonstrations in Section 9)
Assessment	Auto-quiz: 15 MCQs, 40 s each, $+1/-0.25$. Assignment: 5 problems, 100 marks, plus a 20-mark bonus problem.

Overview

Lecture 6 opens the deep-learning block (Lectures 6–9). Part I, *Why Hidden Layers?*, shows that one neuron draws one straight line and therefore cannot compute XOR, that wide or stacked *linear* layers add no power, and that one hidden layer with a non-linear activation solves XOR by re-mapping the data; hidden layers are learned feature maps, and depth builds features of features. Part II, *Backpropagation by Hand*, takes a 2–2–1 sigmoid network with binary cross-entropy through a full forward pass, a full backward pass by the chain rule, a numerical gradient check and one gradient-descent step, ending with the vanishing-gradient argument that motivates ReLU and residual connections. Every later lecture relies on this one: CNNs and Transformers are trained by exactly this backward pass, and the DQN and policy networks of Lectures 10–12 are neural networks too.

Learning outcomes

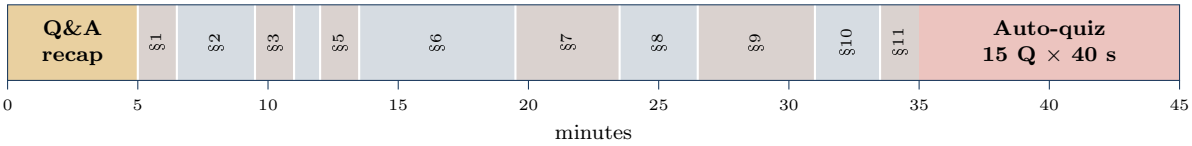
By the end of the lecture students should be able to:

1. Show that a single neuron's decision boundary is a straight line and prove that no single neuron computes XOR.
2. Explain why wide-but-linear and stacked linear layers collapse into one linear map.
3. Compute the forward pass of a ReLU hidden layer that solves XOR, and explain hidden layers as learned feature maps.
4. Carry out a full forward and backward pass of a 2–2–1 sigmoid network with binary cross-entropy, and verify a gradient numerically.
5. Explain the vanishing-gradient problem and why ReLU and residual connections counter it.

Syllabus mapping

- Deep learning foundations — **neural networks**, hidden layers and **backpropagation**

2 Session Plan (45 Minutes)



Clock	Slides	Segment	Min
0:00–5:00	—	**Opening Q&A and recap** (see prompts below)	5
5:00–6:30	2–3	1. Overview (2 slides)	1.5
6:30–9:30	4–7	2. One Neuron (4 slides)	3
9:30–11:00	8–9	3. Wide Is Not Enough (2 slides)	1.5
11:00–12:00	10	4. Non-Linearity (1 slide)	1
12:00–13:30	11–12	5. XOR Solved (2 slides)	1.5
13:30–19:30	13–20	6. Feature Maps (8 slides)	6
19:30–23:30	21–25	7. Backprop: Set-Up (5 slides)	4
23:30–26:30	26–29	8. Forward Pass (4 slides)	3
26:30–31:00	30–35	9. Backward Pass (6 slides)	4.5
31:00–33:30	36–38	10. Update & Check (3 slides)	2.5
33:30–35:00	39–40	11. Summary (2 slides)	1.5
35:00–45:00	—	**Auto-quiz** (instructions 1 min, 15 questions × 40 s = 10 min of answering; collect scores)	10

Slide 1 is the title slide and is shown while students settle. The teaching segment covers 39 content slides in 30 minutes, an average pace of 1.3 slides per minute.

Pitfall — pace

This pace is only achievable if most survey slides get one sentence each. See the facilitation tips for a fast path through a single session and a two-session alternative.

Opening Q&A prompts (first 5 minutes)

1. From Lecture 5: what does an autoencoder learn, and why does a large reconstruction error suggest an anomaly?
2. Logistic regression (Lecture 2) outputs a probability from a weighted sum. What shape is its decision boundary?
3. Which detection problems in the course so far needed a boundary that was not a straight line?

Take two or three answers per prompt; correct misconceptions briefly and note any topic that needs revisiting.

3 Slide-by-Slide Teaching Guide

Each subsection gives the slide range, the suggested time, what to emphasise, and the slide titles in order. Numbers refer to pages of the Beamer PDF.

3.1 1. Overview (slides 2–3, about 1.5 min)

Teaching note. Objectives and the two-part structure; the one-slide story: a line, a wall, a hidden layer, then how the weights are learned.

2 Learning Objectives & Session Structure
3 The Story in One Slide

3.2 2. One Neuron (slides 4–7, about 3 min)

Teaching note. One neuron: weighted sum, bias, activation; the boundary $w_1x_1 + w_2x_2 + b = 0$ is a straight line; AND ($b = -1.5$) and OR ($b = -0.5$) differ only in the bias; the XOR proof by adding inequalities; non-separable patterns in real sensor data.

4 A Single Neuron Draws a Straight Line
5 What One Neuron Can Do: AND and OR
6 The XOR Wall: A Problem No Single Neuron Can Solve
7 XOR Is Not a Toy: Non-Separable Patterns in Real Data

3.3 3. Wide Is Not Enough (slides 8–9, about 1.5 min)

Teaching note. Many neurons in one layer are many lines, each still linear; two linear layers compose into $W_2W_1x + (W_2b_1 + b_2)$, a single linear map.

8 One Layer with Many Neurons Is Still Linear
9 Stacking Linear Layers Collapses into One

3.4 4. Non-Linearity (slide 10, about 1 min)

Teaching note. Step, sigmoid, tanh and ReLU; the sigmoid's slope is at most 0.25, ReLU's is exactly 1 when active. Without a non-linearity, depth is useless.

10 Non-Linear Activation Functions

3.5 5. XOR Solved (slides 11–12, about 1.5 min)

Teaching note. The two-unit ReLU network $h_1 = \mathrm{ReLU}(x_1 + x_2)$, $h_2 = \mathrm{ReLU}(x_1 + x_2 - 1)$, $\hat{y} = h_1 - 2h_2$; work all four rows on the board; show that removing ReLU gives $2 - x_1 - x_2$.

11 A Hidden Layer Solves XOR
12 XOR Solved: The Forward Pass for All Four Inputs

3.6 6. Feature Maps (slides 13–20, about 6 min)

Teaching note. Hidden space: the two XOR-positive inputs land on one point and a single line now separates the classes. Each ReLU adds a bend; the universal approximation theorem and its fine print (existence, not learnability or size); depth as features of features; hidden units as learned indicators; misconceptions; Part I quick check.

13 The Key Insight: Hidden Layers Re-Map the Data
14 Each ReLU Unit Adds a Bend: Building Any Shape
15 The Universal Approximation Theorem — and Its Fine Print
16 Why Depth: Features of Features
17 Feature Maps: The Added Advantage of Hidden Layers
18 Defence Example: Hidden Units as Learned Indicators
19 Common Misconceptions
20 Part 1 Summary and Quick Check — Why Hidden Layers?

3.7 7. Backprop: Set-Up (slides 21–25, about 4 min)

Teaching note. Part II set-up: where backpropagation sits in a defence ML pipeline; the network and its nine parameters; layer-superscript notation; the sigmoid and its derivative; binary cross-entropy.

21 Part 2 — Backpropagation by Hand
22 Where Backpropagation Sits in a Defence ML Pipeline
23 The Network and Its Parameters
24 Notation and the Sigmoid Activation
25 The Loss Function: Binary Cross-Entropy

3.8 8. Forward Pass (slides 26–29, about 3 min)

Teaching note. F1–F5 with all numbers: $z^{(1)} = (0.3, 0.6)$, $a^{(1)} = (0.5744, 0.6457)$, $z^{(2)} = 0.1793$, $a^{(2)} = 0.5447$, $L = 0.6075$.

26 Forward Pass F1 — Hidden Pre-activations $z^{(1)} = W^{(1)}x + b^{(1)}$
27 Forward Pass F2 — Hidden Activations $a_j^{(1)} = \sigma(z_j^{(1)})$
28 Forward Pass F3 & F4 — Output Unit
29 Forward Pass F5 — Loss, and the Forward Summary

3.9 9. Backward Pass (slides 30–35, about 4.5 min)

Teaching note. The chain rule as a computational graph; $\delta^{(2)} = a - y = -0.4553$; output gradients $(-0.2615, -0.2940)$; hidden error signals $(-0.0779,\ 0.0521)$ — note the sign flip through the negative weight; hidden gradients $\delta_j^{(1)} x_k$; the backward-flow diagram.

30 The Chain Rule as a Computational Graph
31 Backward Pass B1 — Output Error Signal $\delta^{(2)}$
32 Backward Pass B2 — Output-Layer Gradients
33 Backward Pass B3 & B4 — Error Signals at the Hidden Layer
34 Backward Pass B5 — Hidden-Layer Gradients
35 Backward-Flow Diagram

3.10 10. Update & Check (slides 36–38, about 2.5 min)

Teaching note. All nine gradients; one step with $\eta = 0.5$ lowers the loss to 0.4377; the numerical check; common mistakes (signs, stale parameters, forgetting σ').

36 All Gradients at a Glance
37 One Gradient-Descent Step ($\eta = 0.5$, assumed) and a Sanity Check
38 Common Mistakes When Computing Backpropagation by Hand

3.11 11. Summary (slides 39–40, about 1.5 min)

Teaching note. The four equations of backpropagation; look-ahead to CNNs.

39 Part 2 Summary — The Four Equations of Backpropagation
40 Looking Ahead — Lecture 7: Convolutional Neural Networks

4 Key Messages

Students should leave the room able to state these in their own words:

Key point

1. One neuron draws one straight line; XOR, and much real data, cannot be separated by one line.
2. Without a non-linear activation, any number of layers is still one linear map.
3. A hidden layer re-maps the data into a space where a line suffices: hidden layers are learned feature maps.
4. Backpropagation is the chain rule applied layer by layer, reusing each layer's error signal; for sigmoid with cross-entropy the output error is simply $a - y$.
5. Each sigmoid layer shrinks the gradient by up to 4 times; ReLU and residual connections keep deep networks trainable.

5 Facilitation Tips

- The deck has 40 slides: teach Part I in about 13 minutes and Part II in about 17. If time is short, show slides 14–18 as one idea ("hidden layers bend and re-map space") and spend the saved minutes on the forward and backward pass.
- Do the XOR proof and the forward pass on the board before showing the slide; students remember what they wrote.
- Use the Neural Network Lab: Demo 1's "search all settings" button makes the impossibility of XOR tangible, and Demo 3 reproduces every number of Part II.

6 Anticipated Questions and Model Answers

Check your understanding — anticipated question

Q: If one hidden layer can approximate any function, why use deep networks?

A: The theorem guarantees existence, not a practical size or a way to learn it. Some functions need exponentially many units in one layer but few in a deep network; depth also reuses features.

Check your understanding — anticipated question

Q: Why not use the step function everywhere?

A: Its derivative is zero almost everywhere, so gradients carry no information; smooth or piecewise-linear activations make gradient descent possible.

Check your understanding — anticipated question

Q: Does the learning rate $\eta = 0.5$ come from the original problem?

A: No; it is assumed for illustration. Any small positive rate reduces the loss here; too large a rate overshoots.

7 Common Misconceptions

Misconception	Correction
"More neurons in one layer can draw a curve."	Each is a line; only a non-linear combination across layers produces curved boundaries.
"Backpropagation is a different learning algorithm from gradient descent."	Backpropagation computes the gradients; gradient descent uses them to update the weights.
"The hidden error signal has the same sign as the output error."	It is multiplied by the outgoing weight, which may be negative, as for $\delta_2^{(1)}$.

8 In-Class Activities and Discussion Prompts

- **XOR in the field:** pairs list three defence detection rules of the form "exactly one of two indicators" (e.g. transponder off but flight plan filed) and explain why a single-layer model fails on them.
- **Backprop relay:** three students compute F1–F5, three compute B1–B5 from the board, one checks with the studio.

9 Run It Live: Studio Demonstrations

Project `Lecture6_Neural_Network_Lab.html` (it is a single HTML file and runs offline in any browser; every result updates as an input changes). Each demonstration below follows the same rhythm: **set** the inputs, **ask** the class to commit to a prediction (a show of hands or a one-word answer), then **reveal** the result and **land** the point. The numbers are exactly what the page shows for the settings given; simulations use fixed random seeds, so they repeat every time. The 45-minute plan has little slack: choose the one demonstration marked *first choice* for class, and use the others in a tutorial, a second session, or as homework with the drill cards at the bottom of the page.

Demo 1: No line computes XOR (about 3 min, with §2 One Neuron) — *first choice*

Set. Demo 1: press AND, then OR, then NAND to show one neuron computing each gate; then ask a volunteer to tilt the line to compute XOR.

Ask. "Can any setting of w_1, w_2 and b get all four XOR points right?"

Reveal. Press Search all settings for XOR: across all 226,981 settings on the slider grid, the fewest errors is **1**. No setting is perfect.

Land the point. One neuron draws one straight line. XOR's positive points sit on opposite corners, so no line separates them: the proof on the slide, confirmed by brute force.

Demo 2: Take the activation away (about 2 min, with §5 XOR Solved)

Set. Demo 2 at its defaults (ReLU, $c_2 = -1$): all four outputs correct, and in hidden space the two XOR-positive inputs land on the same point.

Ask. "If we switch ReLU off, the network keeps both layers. Does it still compute XOR?"

Reveal. Select identity: $\hat{y} = 2 - x_1 - x_2$, only **3 of 4** correct; input $(0, 0)$ now gives 2.

Land the point. Without a non-linearity, two layers collapse into one linear map. The hidden layer's power comes from re-mapping the data non-linearly.

Demo 3: Watch backpropagation learn (about 3 min, with §10 Update & Check)

Set. Demo 3 at its defaults: $x = (1, 0.5)$, $y = 1$, $\eta = 0.5$. The page shows $a^{(2)} = 0.5447$, $L = 0.6075$, $\delta^{(2)} = -0.4553$, matching the slides.

Ask. "After one step, will the loss go down, and to roughly what?"

Reveal. Take one step: $L =$ **0.4377**, $\hat{y} = 0.6455$. Ten steps: after 11 steps $L = 0.0896$, $\hat{y} = 0.9143$. Switch the label to $y = 0$: $L = 0.7868$, and one step gives 0.5454.

Land the point. The gradients point the right way: each step moves the prediction towards the label. The same chain rule trains every network in Lectures 7–12.

Demo 4: Ten sigmoid layers (about 2 min, with §11 Summary)

Set. Demo 4 at its defaults: 10 layers, $|w| = 1$, $z = 0.5$, sigmoid. The factor per layer is 0.2350.

Ask. "What fraction of the output error reaches the first layer?"

Reveal. 5.14×10^{-7}. Switch to ReLU: the factor is 1 and the signal arrives intact; set z negative and ReLU passes nothing (a dead unit).

Land the point. Sigmoid slopes of at most 0.25 make gradients vanish geometrically with depth; ReLU and residual connections fix this.

10 Quiz Administration and Answer Key

Read aloud to students

"You will now take a 15-question quiz. Each question has four options and exactly one correct answer. You have 40 seconds per question; the quiz advances automatically when time runs out. A correct answer scores +1, a wrong answer −0.25, and an unanswered question 0. The maximum is 15 and the minimum is −3.75. Because of the negative marking, guessing blindly among four options gains nothing on average — answer when you can eliminate options. Your score and a full review appear at the end."

Delivery. Open `Lecture6_Quiz.html` on each student device (or project it and collect answers on paper). The page enforces the timer and computes the score with negative marking. Release `Lecture6_Answers_Assignment.html` only after all students have finished.

Answer key

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
B	C	B	A	D	B	B	A	C	A	D	B	A	C	D

Interpreting scores. 12 or more: secure command of the lecture. 9–11.75: good, review missed items. Below 9: revisit the reference text before the assignment. If more than half the class misses a question, revisit that point in the next lecture's opening Q&A.

Questions, options and rationale

Q1. A single neuron with a monotonic activation, thresholded to classify two inputs, always produces a decision boundary that is:

A. A circle
B. A straight line ✓
C. Two parallel lines
D. Any curve, given enough training

Rationale: The rule $\hat{y} = 1 \iff w_1 x_1 + w_2 x_2 + b > 0$ describes the region on one side of a straight line.

Q2. With $w = (1, 1)$, which bias makes one step neuron compute AND on binary inputs?

A. $b = 0.5$
B. $b = -0.5$
C. $b = -1.5$ ✓
D. $b = -2.5$

Rationale: Only $(1, 1)$ gives $1 + 1 - 1.5 > 0$. With $b = -0.5$ the neuron computes OR; with $b = -2.5$ it never fires.

Q3. Why can no single neuron compute XOR?

A. Its bias cannot be negative
B. The two positive points lie on opposite corners, so no straight line separates them from the two negative points ✓
C. XOR needs more than two inputs
D. The sigmoid saturates at 0 and 1

Rationale: Adding the conditions for $(0, 1)$ and $(1, 0)$ gives $w_1 + w_2 + 2b > 0$; those for $(0, 0)$ and $(1, 1)$ give $w_1 + w_2 + 2b \leq 0$. Contradiction.

Q4. Two linear layers $y = W_2(W_1 x + b_1) + b_2$ with no activation between them are equivalent to:

A. One linear layer $y = W_2W_1x + (W_2b_1 + b_2)$ ✓
B. A network with one ReLU hidden layer
C. A deeper and more expressive model
D. A polynomial of degree 2

Rationale: A composition of linear maps is linear: without a non-linearity, depth adds nothing.

Q5. In the network $h_1 = \text{ReLU}(x_1 + x_2)$, $h_2 = \text{ReLU}(x_1 + x_2 - 1)$, $\hat{y} = h_1 - 2h_2$, what is $\hat{y}$ for input $(1, 1)$?

A. 2
B. 1
C. -1
D. 0 ✓

Rationale: $h_1 = 2$, $h_2 = 1$, so $\hat{y} = 2 - 2 = 0$: the correct XOR output.

Q6. In the same XOR network with ReLU replaced by the identity, the outputs for $(0, 0), (0, 1), (1, 0), (1, 1)$ become:

A. 0, 1, 1, 0
B. 2, 1, 1, 0 ✓
C. 0, 1, 1, 2
D. 1, 1, 1, 1

Rationale: $\hat{y} = (x_1 + x_2) - 2(x_1 + x_2 - 1) = 2 - x_1 - x_2$: a single linear function, which fails at $(0, 0)$.

Q7. How many parameters does a dense network $20 \to 64 \to 32 \to 5$ have?

A. 3,328
B. 3,589 ✓
C. 3,488
D. 4,096

Rationale: $21 \times 64 + 65 \times 32 + 33 \times 5 = 1{,}344 + 2{,}080 + 165 = 3{,}589$.

Q8. A one-hidden-layer network on a single input with n ReLU units produces a piecewise-linear function with at most how many bends?

A. n ✓
B. $2n$
C. n^2
D. 2^n

Rationale: Each ReLU contributes one kink where its input crosses zero. Depth, not width, gives exponentially many pieces.

Q9. The derivative of the sigmoid can be written as:

A. $\sigma(z)^2$
B. $1 - \sigma(z)$
C. $\sigma(z)(1 - \sigma(z))$ ✓
D. e^{-z}

Rationale: Its maximum is 0.25, at $z = 0$.

Q10. For a sigmoid output with binary cross-entropy loss, the output error signal $\partial L/\partial z^{(2)}$ simplifies to:

A. $a^{(2)} - y$ ✓
B. $(a^{(2)} - y)^2$
C. $-\ln a^{(2)}$
D. $a^{(2)}(1 - a^{(2)})$

Rationale: The σ' factor of the chain rule cancels against the loss derivative, leaving prediction minus label.

Q11. In the lecture's 2–2–1 example, $a^{(2)} = 0.5447$ and $y = 1$. The loss $L = -\ln a^{(2)}$ is about:

A. 0.455
B. 0.788
C. 0.545
D. 0.608 ✓

Rationale: $-\ln 0.5447 = 0.6075$.

Q12. The hidden error signal is $\delta_j^{(1)} = \delta^{(2)}\, w_j^{(2)}\, \sigma'(z_j^{(1)})$. In the lecture's example, $\delta_2^{(1)}$ is positive while $\delta^{(2)}$ is negative because:

A. σ' is negative
B. $w_2^{(2)} = -0.5$ is negative ✓
C. x_2 is smaller than x_1
D. The bias $b^{(2)}$ is positive

Rationale: $-0.4553 \times (-0.5) \times 0.2288 = +0.0521$: increasing $a_2^{(1)}$ would lower the output, so the gradient pushes it the other way.

Q13. A numerical gradient check estimates $\partial L/\partial\theta$ by:

A. $[L(\theta + h) - L(\theta - h)]/(2h)$ for a small h ✓
B. Setting $\theta = 0$ and reading L
C. Averaging the gradients of all parameters
D. Running backpropagation twice

Rationale: The central difference agrees with backpropagation to many decimal places when the code is correct.

Q14. With $\eta = 0.5$, the gradient-descent step in the lecture lowered the loss from 0.6075 to:

A. 0.5447
B. 0.6455
C. 0.4377 ✓
D. 0.3276

Rationale: After the update $\hat{y} = 0.6455$ and $L = -\ln 0.6455 = 0.4377$.

Q15. Ten stacked sigmoid layers shrink the error signal reaching the first layer by a factor of at least about:

A. 10^{-3}
B. 2.5
C. 0.25
D. 10^{-6} ✓

Rationale: $\sigma' \le 0.25$, so $0.25^{10} \approx 9.5 \times 10^{-7}$. This vanishing gradient is why ReLU and residual connections are used.

11 Assignment: Marking Scheme and Model Solutions

Instructions. Five problems, 100 marks, plus a 20-mark bonus problem (Problem 6). Reference: Lecture 6 reference text.

#	Problem	Marks
1	Single-neuron boundaries and the XOR wall	20
2	A ReLU hidden layer: forward pass and collapse	20
3	A full sigmoid forward pass	20
4	The backward pass	20
5	One update and the vanishing gradient	20
6	Problem 6 (bonus): Parameter counting and depth	20
	Total	**100** + 20 bonus

General marking guidance

- Marks for each part are shown in square brackets in the problem statement.
- Award method marks for correct working even when an arithmetic slip carries through; deduct once for the slip, not again for its consequences.
- For interpretive parts, full marks need (i) the correct conclusion, (ii) the reason, and (iii) an operational implication or caveat.
- Answers that state a number without uncertainty, baseline or caveat where one is required lose a mark per part.
- Suggested deadline: one week after the lecture; return marked work before the next quiz on the same topic.

11.1 Problem 1: Single-neuron boundaries and the XOR wall (20 marks)

Problem statement

A step neuron outputs $\hat{y} = 1$ if $w_1x_1 + w_2x_2 + b > 0$ and 0 otherwise, on binary inputs.
(a) With $w = (1, 1)$, give a bias for AND and a bias for OR, and verify each on all four inputs. [5]
(b) Find weights and a bias for NAND (output 0 only for $(1, 1)$). [4]
(c) Prove that no single neuron computes XOR. [6]
(d) Show that XOR = AND(OR, NAND), and hence that a network with one hidden layer of two step neurons computes XOR. [5]

Model solution and marking points

(a) AND: $b = -1.5$. Sums $x_1 + x_2 - 1.5$ are $-1.5, -0.5, -0.5, 0.5$ for $(0, 0), (0, 1), (1, 0), (1, 1)$, so only $(1, 1)$ fires. OR: $b = -0.5$, sums $-0.5, 0.5, 0.5, 1.5$, so all but $(0, 0)$ fire. Any $b \in (-2, -1]$ works for AND and any $b \in (-1, 0]$ for OR (with the strict inequality > 0).
(b) NAND = NOT AND: negate the AND neuron. $w = (-1, -1)$, $b = 1.5$: sums $1.5, 0.5, 0.5, -0.5$, so it fires except at $(1, 1)$.
(c) Suppose w_1, w_2, b compute XOR. $(0, 0) \mapsto 0$: $b \leq 0$. $(1, 1) \mapsto 0$: $w_1 + w_2 + b \leq 0$. $(0, 1) \mapsto 1$: $w_2 + b > 0$. $(1, 0) \mapsto 1$: $w_1 + b > 0$. Adding the last two: $w_1 + w_2 + 2b > 0$. Adding the first two: $w_1 + w_2 + 2b \leq 0$. Contradiction, so no such neuron exists. Geometrically the positive

points sit on opposite corners of the square and no straight line separates them from the other two.
(d) Table: $(0,0)$: OR 0, NAND 1, AND 0. $(0,1)$ and $(1,0)$: OR 1, NAND 1, AND 1. $(1,1)$: OR 1, NAND 0, AND 0. This is XOR. Network: hidden unit h_1 = OR ($w=(1,1)$, $b=-0.5$), hidden unit h_2 = NAND ($w=(-1,-1)$, $b=1.5$), output = AND of h_1, h_2 ($w=(1,1)$, $b=-1.5$). The hidden layer re-maps the four points so that the output's single line can separate them.

11.2 Problem 2: A ReLU hidden layer: forward pass and collapse (20 marks)

Problem statement

The network $h_1 = \mathrm{ReLU}(x_1+x_2)$, $h_2 = \mathrm{ReLU}(x_1+x_2-1)$, $\hat{y} = h_1 - 2h_2$ is the lecture's XOR solver.
(a) Tabulate $z_1, z_2, h_1, h_2, \hat{y}$ for all four binary inputs. [6]
(b) Replace ReLU by the identity, simplify $\hat{y}$, and show it fails. [5]
(c) For one input x, evaluate $f(x) = \mathrm{ReLU}(x) - 2\,\mathrm{ReLU}(x-1) + \mathrm{ReLU}(x-2)$ at $x = -1, 0, 0.5, 1, 1.5, 2, 3$ and sketch it. How many bends has it, and why? [6]
(d) Count the parameters of the XOR network (no output bias). [3]

Model solution and marking points

(a) $(0,0)$: $z=(0,-1)$, $h=(0,0)$, $\hat{y}=0$. $(0,1)$ and $(1,0)$: $z=(1,0)$, $h=(1,0)$, $\hat{y}=1$. $(1,1)$: $z=(2,1)$, $h=(2,1)$, $\hat{y}=2-2=0$. All correct. h_1 is an OR-like counter, h_2 an AND detector: XOR = OR and not AND.
(b) $\hat{y} = (x_1+x_2) - 2(x_1+x_2-1) = 2 - x_1 - x_2$, giving $2, 1, 1, 0$. The output at $(0,0)$ is 2, not 0. Two linear layers collapse into one linear function, which cannot compute XOR.
(c) $f(-1)=0$, $f(0)=0$, $f(0.5)=0.5$, $f(1)=1$, $f(1.5)=0.5$, $f(2)=0$, $f(3)=3-4+1=0$. It is a triangular bump on $[0,2]$ with peak 1 at $x=1$. Three bends, at $x=0,1,2$, one per ReLU unit, where each unit's input crosses zero. Sums of such bumps approximate any continuous function on an interval: the idea behind the universal approximation theorem.
(d) Hidden layer: 2×2 weights + 2 biases = 6; output: 2 weights. Total **8**.

11.3 Problem 3: A full sigmoid forward pass (20 marks)

Problem statement

Use the lecture's 2–2–1 sigmoid network: $x=(1.0, 0.5)$, $W^{(1)} = \begin{pmatrix} 0.4 & -0.2 \\ 0.3 & 0.6 \end{pmatrix}$, $b^{(1)} = (0,0)$, $w^{(2)} = (0.7, -0.5)$, $b^{(2)} = 0.1$, binary cross-entropy loss. This time the correct label is $\mathbf{y} = \mathbf{0}$ (the track is *not* hostile).
(a) Compute $z^{(1)}$ and $a^{(1)}$. [6]
(b) Compute $z^{(2)}$ and the prediction $a^{(2)}$. [6]
(c) Compute the loss and compare it with the lecture's loss for $y=1$. [5]
(d) Explain in one sentence why the forward pass does not depend on the label. [3]

Model solution and marking points

(a) $z_1^{(1)} = 0.4(1) - 0.2(0.5) = 0.3$; $z_2^{(1)} = 0.3(1) + 0.6(0.5) = 0.6$. $a^{(1)} = (\sigma(0.3), \sigma(0.6)) =$ (**0.5744**, **0.6457**).

(b) $z^{(2)} = 0.7(0.5744) - 0.5(0.6457) + 0.1 = 0.4021 - 0.3229 + 0.1 = \mathbf{0.1793}$; $a^{(2)} = \sigma(0.1793) = \mathbf{0.5447}$.
(c) $L = -[y \ln a + (1-y)\ln(1-a)] = -\ln(1-0.5447) = -\ln 0.4553 = \mathbf{0.7868}$. For $y = 1$ the loss was $-\ln 0.5447 = 0.6075$. The network leans slightly towards "hostile", so it is penalised more when the truth is "not hostile".
(d) The forward pass computes the prediction from the inputs and weights only; the label enters only in the loss.

11.4 Problem 4: The backward pass (20 marks)

Problem statement

Continue Problem 3 with $y = 0$.
(a) Compute the output error signal $\delta^{(2)} = \partial L/\partial z^{(2)}$ and the gradients for $w^{(2)}$ and $b^{(2)}$. [6]
(b) Compute $\sigma'(z_j^{(1)})$ and the hidden error signals $\delta_j^{(1)}$. [6]
(c) Compute the gradients for all four entries of $W^{(1)}$ and for $b^{(1)}$. [5]
(d) Check $\partial L/\partial w_1^{(2)}$ numerically with $h = 10^{-4}$. [3]

Model solution and marking points

(a) $\delta^{(2)} = a^{(2)} - y = \mathbf{0.5447}$. $\partial L/\partial w^{(2)} = \delta^{(2)} a^{(1)} = (0.5447 \times 0.5744,\ 0.5447 \times 0.6457) = (\mathbf{0.3129},\ \mathbf{0.3517})$; $\partial L/\partial b^{(2)} = \mathbf{0.5447}$.
(b) $\sigma' = a(1-a)$: $(0.5744 \times 0.4256,\ 0.6457 \times 0.3543) = (0.2445,\ 0.2288)$. $\delta_1^{(1)} = 0.5447 \times 0.7 \times 0.2445 = \mathbf{0.0932}$; $\delta_2^{(1)} = 0.5447 \times (-0.5) \times 0.2288 = \mathbf{-0.0623}$.
(c) $\partial L/\partial w_{jk}^{(1)} = \delta_j^{(1)} x_k$: $\begin{pmatrix} 0.0932 & 0.0466 \\ -0.0623 & -0.0312 \end{pmatrix}$; $\partial L/\partial b^{(1)} = (0.0932,\ -0.0623)$. All signs are the opposite of the lecture's $y = 1$ case, as expected: the error $a - y$ changed sign.
(d) $[L(0.7001) - L(0.6999)]/0.0002 = 0.3129$, agreeing with backpropagation to four decimals.

11.5 Problem 5: One update and the vanishing gradient (20 marks)

Problem statement

(a) Apply one gradient-descent step with $\eta = 0.5$ to every parameter, using the gradients of Problem 4. [6]
(b) Repeat the forward pass with the new parameters. Did the prediction move towards $y = 0$, and by how much did the loss change? [6]
(c) A network has ten sigmoid hidden layers. Give a lower bound on the factor by which the error signal can shrink between the output and the first layer (weights of size about 1), and explain the consequence. [5]
(d) Name two design choices that counter this and say why each works. [3]

Model solution and marking points

(a) $w^{(2)} \leftarrow (0.7 - 0.1564,\ -0.5 - 0.1758) = (\mathbf{0.5436},\ \mathbf{-0.6758})$; $b^{(2)} \leftarrow 0.1 - 0.2724 = \mathbf{-0.1724}$.
$W^{(1)} \leftarrow \begin{pmatrix} 0.3534 & -0.2233 \\ 0.3312 & 0.6156 \end{pmatrix}$; $b^{(1)} \leftarrow (-0.0466,\ 0.0312)$.
(b) $a^{(1)} = (\sigma(0.1952), \sigma(0.6702)) = (0.5486,\ 0.6615)$; $z^{(2)} = 0.5436(0.5486) - 0.6758(0.6615) - 0.1724 = -0.3212$; $a^{(2)} = \mathbf{0.4204}$. The prediction fell from 0.5447 towards 0, and the loss fell from 0.7868 to $-\ln(0.5796) = \mathbf{0.5454}$.

(c) Each layer multiplies the signal by a weight and by $\sigma'(z) \le 0.25$, so ten layers shrink it by at least $0.25^{10} \approx \mathbf{9.5 \times 10^{-7}}$. The first layers receive almost no gradient and barely learn: the vanishing-gradient problem.
(d) ReLU hidden units: derivative exactly 1 when active, so active paths do not shrink the signal. Residual connections $y = x + F(x)$: $\partial y/\partial x = I + \partial F/\partial x$ gives a direct path for the gradient. (Careful initialisation and normalisation layers also help.)

11.6 Problem 6 (bonus): Parameter counting and depth (20 marks)

Problem statement

(a) Count the parameters of dense networks 20–64–32–5 and 784–128–64–10. [6]
(b) A one-hidden-layer network 20–m–5 is to have about the same number of parameters as 20–64–32–5. Find m. [4]
(c) The "tent" $g(x) = 2\,\mathrm{ReLU}(x) - 4\,\mathrm{ReLU}(x - 0.5)$ maps $[0, 1]$ onto $[0, 1]$ with one peak. Composing it k times gives 2^{k-1} peaks. How many ReLU units does the k-fold composition use, and how many would one hidden layer need for the same number of linear pieces? What does this say about depth? [10]

Model solution and marking points

(a) $21 \times 64 + 65 \times 32 + 33 \times 5 = 1{,}344 + 2{,}080 + 165 = \mathbf{3{,}589}$. $785 \times 128 + 129 \times 64 + 65 \times 10 = 100{,}480 + 8{,}256 + 650 = \mathbf{109{,}386}$.
(b) $21m + (m + 1) \times 5 = 26m + 5 = 3{,}589 \Rightarrow m \approx 137.8$; $m = 138$ gives 3,593. One wide layer can match the count, but not the hierarchy of features of features.
(c) Each composition uses 2 ReLUs, so k layers use $2k$ units and produce 2^{k-1} peaks, that is 2^k linear pieces. One hidden layer of n ReLUs gives at most $n + 1$ pieces, so it needs about $2^k - 1$ units. For $k = 10$: 20 units in depth against about 1,023 in width. Depth can represent some functions exponentially more efficiently than width: the formal reason for "features of features".